

人工智能赋能的数字集成电路设计作业三

陈天乐

2025 年 11 月 19 日

目录

1 功能一：代码补全	2
2 功能二：代码纠错	6
3 功能三：在聊天框进行代码纠错	12
4 功能四：基于 Specification 的 RTL 生成	16
5 功能五：TestBench 生成及仿真结果	21
6 功能六：时序和面积优化	33
7 对合见工软 UDA 的使用评价，修改建议	38

1 功能一：代码补全

将代码选中，右键选择 Generate Code 的功能。

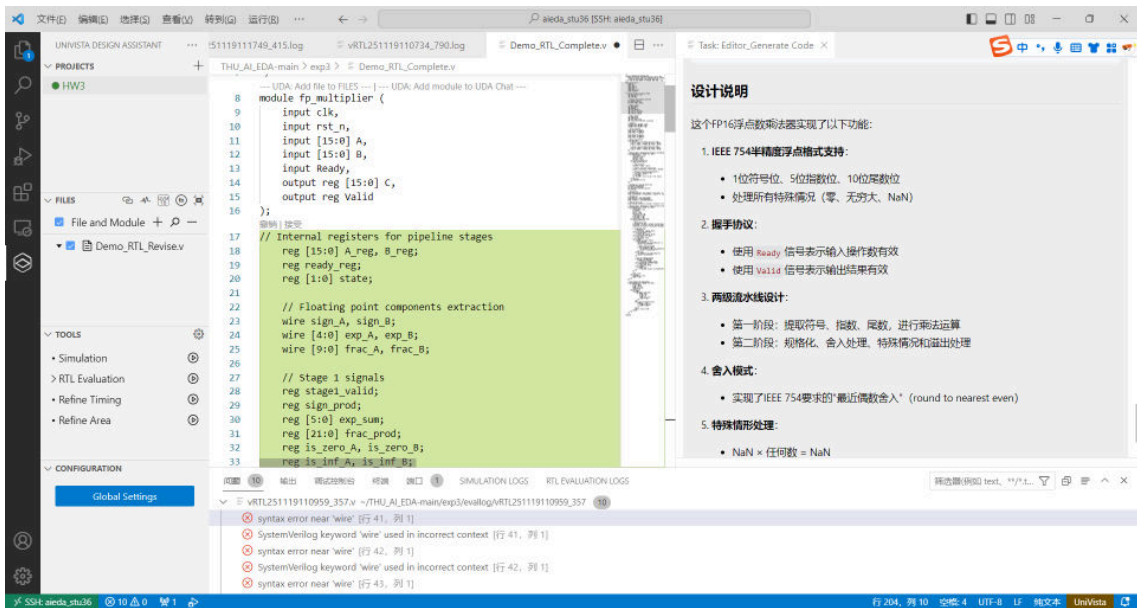


图 1: 代码补全

生成的代码如下：

```
1 /*
2 This is a multiplier for floating point
3 - Support IEEE 754 Half-Precision Floating-Point Format (FP16):
4   - Handle all special cases (zero, infinity, NaN) as specified by IEEE 754.
5   - Ensure correct rounding to the nearest even value.
6 - Use valid and ready signals for handshake. When valid is high, the input operands A and B
   are ready. When ready is high, the result C is valid.
7 */
8
9 module fp_multiplier (
10     input clk,
11     input rst_n,
12     input [15:0] A,
13     input [15:0] B,
14     input Ready,
15     output reg [15:0] C,
16     output reg Valid
17 );
18 // Internal registers for pipeline stages
19 reg [15:0] A_reg, B_reg;
20 reg ready_reg;
21 reg [1:0] state;
22
23 // Floating point components extraction
24 wire sign_A, sign_B;
25 wire [4:0] exp_A, exp_B;
26 wire [9:0] frac_A, frac_B;
27
28 // Stage 1 signals
29 reg stage1_valid;
30 reg sign_prod;
31 reg [5:0] exp_sum;
```

```

32 reg [21:0] frac_prod;
33 reg is_zero_A, is_zero_B;
34 reg is_inf_A, is_inf_B;
35 reg is_nan_A, is_nan_B;
36
37 // Stage 2 signals
38 reg stage2_valid;
39 reg [4:0] exp_adj;
40 reg [10:0] frac_rounded;
41 reg is_zero_prod, is_inf_prod, is_nan_prod;
42
43 // Constants
44 localparam [4:0] EXP_BIAS = 5'd15;
45 localparam [4:0] EXP_MAX = 5'd31;
46 localparam [4:0] EXP_MIN = 5'd0;
47
48 // Extract components
49 assign sign_A = A_reg[15];
50 assign exp_A = A_reg[14:10];
51 assign frac_A = A_reg[9:0];
52
53 assign sign_B = B_reg[15];
54 assign exp_B = B_reg[14:10];
55 assign frac_B = B_reg[9:0];
56
57 // Special case detection
58 always @(*) begin
59     is_zero_A = (exp_A == 5'b0) && (frac_A == 10'b0);
60     is_inf_A = (exp_A == 5'b11111) && (frac_A == 10'b0);
61     is_nan_A = (exp_A == 5'b11111) && (frac_A != 10'b0);
62
63     is_zero_B = (exp_B == 5'b0) && (frac_B == 10'b0);
64     is_inf_B = (exp_B == 5'b11111) && (frac_B == 10'b0);
65     is_nan_B = (exp_B == 5'b11111) && (frac_B != 10'b0);
66 end
67
68 // Pipeline stage 1: Multiplication and special case handling
69 always @(posedge clk or negedge rst_n) begin
70     if (!rst_n) begin
71         stage1_valid <= 1'b0;
72         sign_prod <= 1'b0;
73         exp_sum <= 6'b0;
74         frac_prod <= 22'b0;
75     end else if (ready_reg) begin
76         stage1_valid <= 1'b1;
77
78         // Sign calculation
79         sign_prod <= sign_A ^ sign_B;
80
81         // Exponent calculation (subtract bias)
82         exp_sum <= {1'b0, exp_A} + {1'b0, exp_B} - EXP_BIAS;
83
84         // Fraction multiplication (with implicit 1)
85         if (is_zero_A || is_zero_B) begin
86             frac_prod <= 22'b0;
87         end else begin
88             frac_prod <= {1'b1, frac_A} * {1'b1, frac_B};
89         end
90     end else begin
91         stage1_valid <= 1'b0;
92     end
93 end

```

```

94
95 // Rounding logic (round to nearest even)
96 wire [21:0] frac_normalized;
97 wire [4:0] exp_normalized;
98 wire round_bit, sticky_bit, guard_bit;
99 wire round_up;
100
101 // Normalization
102 assign frac_normalized = (frac_prod[21]) ? (frac_prod >> 1) : frac_prod;
103 assign exp_normalized = (frac_prod[21]) ? (exp_sum + 1) : exp_sum;
104
105 // Rounding bits
106 assign guard_bit = frac_normalized[10];
107 assign round_bit = frac_normalized[9];
108 assign sticky_bit = |frac_normalized[8:0];
109
110 // Round to nearest even
111 assign round_up = guard_bit & (round_bit | sticky_bit | frac_normalized[11]);
112
113 // Pipeline stage 2: Normalization, rounding and final result assembly
114 always @(posedge clk or negedge rst_n) begin
115     if (!rst_n) begin
116         stage2_valid <= 1'b0;
117         C <= 16'b0;
118         Valid <= 1'b0;
119         exp_adj <= 5'b0;
120         frac_rounded <= 11'b0;
121         is_zero_prod <= 1'b0;
122         is_inf_prod <= 1'b0;
123         is_nan_prod <= 1'b0;
124     end else if (stage1_valid) begin
125         stage2_valid <= 1'b1;
126
127         // Handle special cases
128         is_nan_prod <= is_nan_A | is_nan_B | (is_inf_A & is_zero_B) | (is_zero_A &
is_inf_B);
129         is_inf_prod <= (is_inf_A | is_inf_B) & ~is_nan_prod;
130         is_zero_prod <= (is_zero_A | is_zero_B) & ~is_nan_prod;
131
132         if (is_nan_prod) begin
133             // NaN result
134             C <= {sign_prod, 5'b11111, 10'b1};
135         end else if (is_inf_prod) begin
136             // Infinity result
137             C <= {sign_prod, 5'b11111, 10'b0};
138         end else if (is_zero_prod) begin
139             // Zero result
140             C <= {sign_prod, 15'b0};
141         end else begin
142             // Normal case: apply rounding
143             if (round_up) begin
144                 frac_rounded <= frac_normalized[21:11] + 1;
145                 exp_adj <= exp_normalized;
146
147                 // Handle carry from rounding
148                 if (frac_rounded[10]) begin
149                     frac_rounded <= {1'b0, frac_rounded[10:1]};
150                     exp_adj <= exp_normalized + 1;
151                 end
152             end else begin
153                 frac_rounded <= frac_normalized[21:11];
154                 exp_adj <= exp_normalized;

```

```

155         end
156
157         // Check for overflow/underflow
158         if (exp_adj >= EXP_MAX) begin
159             // Overflow to infinity
160             C <= {sign_prod, 5'b11111, 10'b0};
161         end else if (exp_adj <= EXP_MIN) begin
162             // Underflow to zero
163             C <= {sign_prod, 15'b0};
164         end else begin
165             // Normal result
166             C <= {sign_prod, exp_adj, frac_rounded[9:0]};
167         end
168     end
169
170     Valid <= 1'b1;
171 end else begin
172     stage2_valid <= 1'b0;
173     Valid <= 1'b0;
174 end
175 end
176
177 // Input register and ready handling
178 always @(posedge clk or negedge rst_n) begin
179     if (!rst_n) begin
180         A_reg <= 16'b0;
181         B_reg <= 16'b0;
182         ready_reg <= 1'b0;
183         state <= 2'b0;
184     end else begin
185         case (state)
186             2'b00: begin // Idle state
187                 if (Ready) begin
188                     A_reg <= A;
189                     B_reg <= B;
190                     ready_reg <= 1'b1;
191                     state <= 2'b01;
192                 end
193             end
194             2'b01: begin // Processing state
195                 ready_reg <= 1'b0;
196                 if (stage2_valid) begin
197                     state <= 2'b00;
198                 end
199             end
200             default: state <= 2'b00;
201         endcase
202     end
203 end
204
205 endmodule

```

Listing 1: Demo_RTL_Complete.v

2 功能二：代码纠错

将代码选中，右键选择 Fix Code With Error Info 的功能。

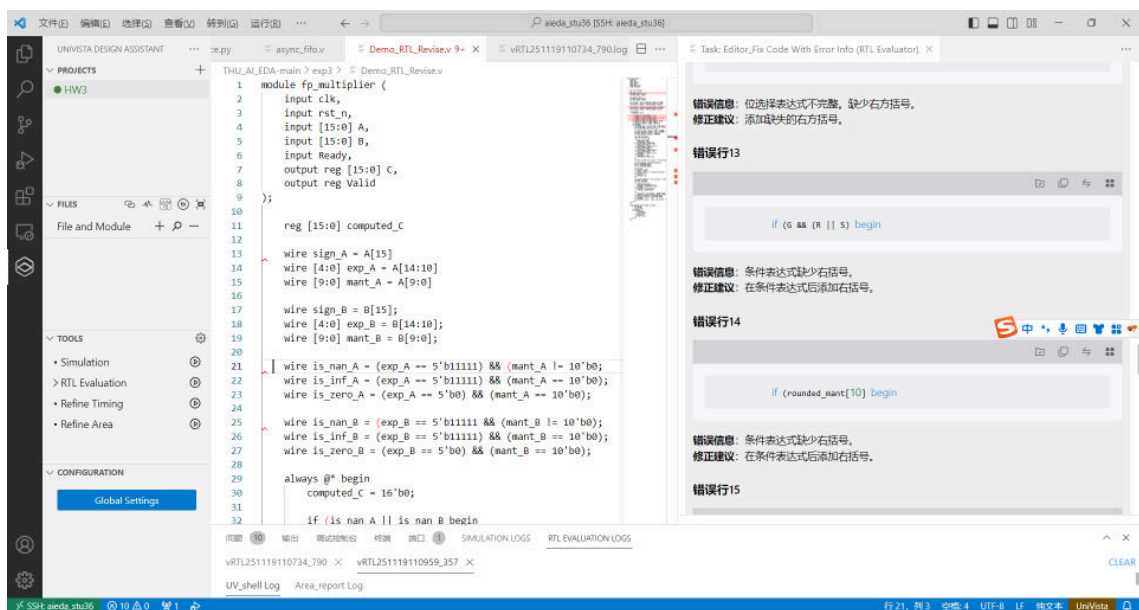
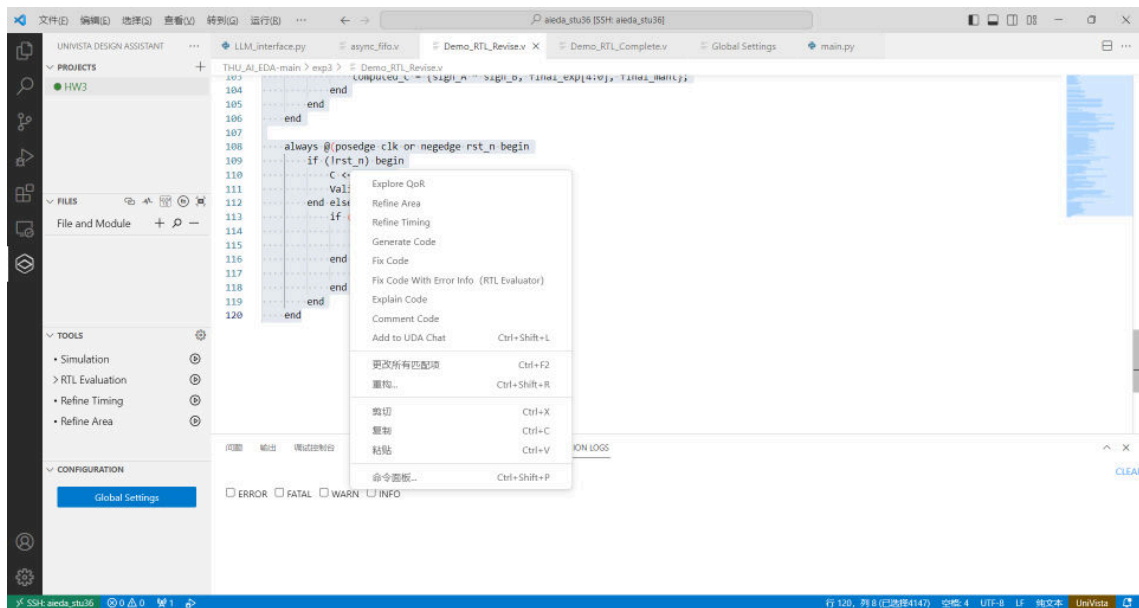


图 2: 代码纠错

修改后的代码如下：

```
1 module fp_multiplier (
2     input clk,
3     input rst_n,
4     input [15:0] A,
5     input [15:0] B,
6     input Ready,
7     output reg [15:0] C,
8     output reg Valid
9 );
10
11 reg [15:0] computed_C;
```

```

12
13 wire sign_A = A[15];
14 wire [4:0] exp_A = A[14:10];
15 wire [9:0] mant_A = A[9:0];
16
17 wire sign_B = B[15];
18 wire [4:0] exp_B = B[14:10];
19 wire [9:0] mant_B = B[9:0];
20
21 wire is_nan_A = (exp_A == 5'b11111) && (mant_A != 10'b0);
22 wire is_inf_A = (exp_A == 5'b11111) && (mant_A == 10'b0);
23 wire is_zero_A = (exp_A == 5'b0) && (mant_A == 10'b0);
24
25 wire is_nan_B = (exp_B == 5'b11111) && (mant_B != 10'b0);
26 wire is_inf_B = (exp_B == 5'b11111) && (mant_B == 10'b0);
27 wire is_zero_B = (exp_B == 5'b0) && (mant_B == 10'b0);
28
29 always @(*) begin
30     computed_C = 16'b0;
31
32     if (is_nan_A || is_nan_B) begin
33         computed_C = {1'b0, 5'b11111, 10'b1};
34     end else if ((is_inf_A && is_zero_B) || (is_zero_A && is_inf_B)) begin
35         computed_C = {1'b0, 5'b11111, 10'b1};
36     end else if (is_inf_A || is_inf_B) begin
37         computed_C = {sign_A ^ sign_B, 5'b11111, 10'b0};
38     end else if (is_zero_A || is_zero_B) begin
39         computed_C = {sign_A ^ sign_B, 5'b0, 10'b0};
40     end else begin
41         wire [10:0] sig_A = (exp_A == 5'b0) ? {1'b0, mant_A} : {1'b1, mant_A};
42         wire [10:0] sig_B = (exp_B == 5'b0) ? {1'b0, mant_B} : {1'b1, mant_B};
43         wire [21:0] product_sig = sig_A * sig_B;
44
45         wire [5:0] exp_A_adj = (exp_A == 5'b0) ? 6'b100001 : {1'b0, exp_A};
46         wire [5:0] exp_B_adj = (exp_B == 5'b0) ? 6'b100001 : {1'b0, exp_B};
47         wire [6:0] exp_sum = exp_A_adj + exp_B_adj;
48         wire [6:0] exp_prod = exp_sum - 7'b0011110;
49
50         reg [4:0] shift_amount;
51         reg [21:0] normalized_sig;
52
53         if (product_sig[21]) begin
54             shift_amount = 5'b0;
55             normalized_sig = product_sig;
56         end else if (product_sig[20]) begin
57             shift_amount = 5'b1;
58             normalized_sig = product_sig << 1;
59         end else if (product_sig[19]) begin
60             shift_amount = 5'b10;
61             normalized_sig = product_sig << 2;
62         end else if (product_sig[18]) begin
63             shift_amount = 5'b11;
64             normalized_sig = product_sig << 3;
65         end else begin
66             shift_amount = 5'b0;
67             normalized_sig = product_sig;
68         end
69
70         wire [6:0] exp_after_norm = exp_prod - {2'b0, shift_amount};
71
72         wire [9:0] mant_part = normalized_sig[20:11];
73         wire G = normalized_sig[10];

```

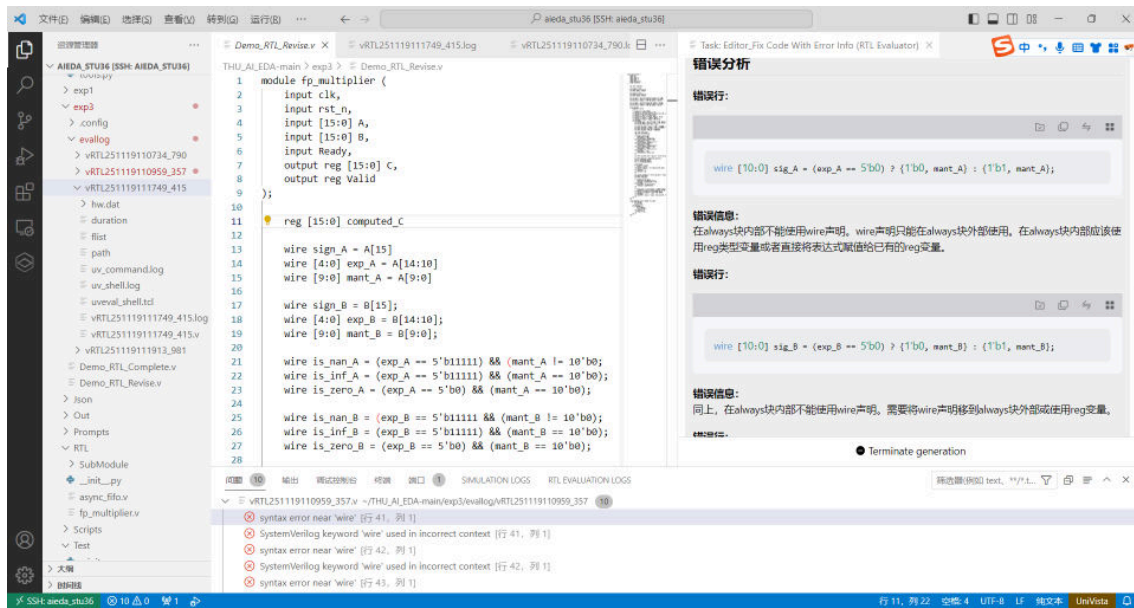
```

74     wire R = normalized_sig[9];
75     wire S = |normalized_sig[8:0];
76
77     wire round_up;
78     if (G && (R || S)) begin
79         round_up = 1'b1;
80     end else if (G && !(R || S) && mant_part[0]) begin
81         round_up = 1'b1;
82     end else begin
83         round_up = 1'b0;
84     end
85
86     wire [10:0] rounded_mant = {1'b0, mant_part} + round_up;
87     wire [9:0] final_mant;
88     wire [6:0] final_exp;
89
90     if (rounded_mant[10]) begin
91         final_mant = rounded_mant[10:1];
92         final_exp = exp_after_norm + 7'b1;
93     end else begin
94         final_mant = rounded_mant[9:0];
95         final_exp = exp_after_norm;
96     end
97
98     if (final_exp[6] || final_exp[5:0] > 6'b001111) begin
99         computed_C = {sign_A ^ sign_B, 5'b11111, 10'b0};
100    end else if (final_exp[5:0] < 6'b000001) begin
101        computed_C = {sign_A ^ sign_B, 5'b0, 10'b0};
102    end else begin
103        computed_C = {sign_A ^ sign_B, final_exp[4:0], final_mant};
104    end
105    end
106    end
107
108    always @(posedge clk or negedge rst_n) begin
109        if (!rst_n) begin
110            C <= 16'b0;
111            Valid <= 1'b0;
112        end else begin
113            if (Ready) begin
114                C <= computed_C;
115                Valid <= 1'b1;
116            end else begin
117                Valid <= 1'b0;
118            end
119        end
120    end
121 endmodule

```

Listing 2: vRTL251119111749_415.v

对这个修改过的代码进行检查，发现还是有些地方出错，比如在 `always` 中使用 `wire`。因此再对代码进行二次修改：



第二次修改的结果如下：

```

1 module fp_multiplier (
2     input clk,
3     input rst_n,
4     input [15:0] A,
5     input [15:0] B,
6     input Ready,
7     output reg [15:0] C,
8     output reg Valid
9 );
10
11 reg [15:0] computed_C;
12
13 wire sign_A = A[15];
14 wire [4:0] exp_A = A[14:10];
15 wire [9:0] mant_A = A[9:0];
16
17 wire sign_B = B[15];
18 wire [4:0] exp_B = B[14:10];
19 wire [9:0] mant_B = B[9:0];
20
21 wire is_nan_A = (exp_A == 5'b11111) && (mant_A != 10'b0);
22 wire is_inf_A = (exp_A == 5'b11111) && (mant_A == 10'b0);
23 wire is_zero_A = (exp_A == 5'b0) && (mant_A == 10'b0);
24
25 wire is_nan_B = (exp_B == 5'b11111) && (mant_B != 10'b0);
26 wire is_inf_B = (exp_B == 5'b11111) && (mant_B == 10'b0);
27 wire is_zero_B = (exp_B == 5'b0) && (mant_B == 10'b0);
28
29 // 将原来在always块内部的wire声明移到外部
30 wire [10:0] sig_A = (exp_A == 5'b0) ? {1'b0, mant_A} : {1'b1, mant_A};
31 wire [10:0] sig_B = (exp_B == 5'b0) ? {1'b0, mant_B} : {1'b1, mant_B};
32 wire [21:0] product_sig = sig_A * sig_B;
33 wire [5:0] exp_A_adj = (exp_A == 5'b0) ? 6'b100001 : {1'b0, exp_A};
34 wire [5:0] exp_B_adj = (exp_B == 5'b0) ? 6'b100001 : {1'b0, exp_B};
35 wire [6:0] exp_sum = exp_A_adj + exp_B_adj;
36 wire [6:0] exp_prod = exp_sum - 7'b0011110;
37
38 always @(*) begin
39     computed_C = 16'b0;
40

```

```

41     if (is_nan_A || is_nan_B) begin
42         computed_C = {1'b0, 5'b11111, 10'b1};
43     end else if ((is_inf_A && is_zero_B) || (is_zero_A && is_inf_B)) begin
44         computed_C = {1'b0, 5'b11111, 10'b1};
45     end else if (is_inf_A || is_inf_B) begin
46         computed_C = {sign_A ^ sign_B, 5'b11111, 10'b0};
47     end else if (is_zero_A || is_zero_B) begin
48         computed_C = {sign_A ^ sign_B, 5'b0, 10'b0};
49     end else begin
50         reg [4:0] shift_amount;
51         reg [21:0] normalized_sig;
52
53         if (product_sig[21]) begin
54             shift_amount = 5'b0;
55             normalized_sig = product_sig;
56         end else if (product_sig[20]) begin
57             shift_amount = 5'b1;
58             normalized_sig = product_sig << 1;
59         end else if (product_sig[19]) begin
60             shift_amount = 5'b10;
61             normalized_sig = product_sig << 2;
62         end else if (product_sig[18]) begin
63             shift_amount = 5'b11;
64             normalized_sig = product_sig << 3;
65         end else begin
66             shift_amount = 5'b0;
67             normalized_sig = product_sig;
68         end
69
70         wire [6:0] exp_after_norm = exp_prod - {2'b0, shift_amount};
71
72         wire [9:0] mant_part = normalized_sig[20:11];
73         wire G = normalized_sig[10];
74         wire R = normalized_sig[9];
75         wire S = |normalized_sig[8:0];
76
77         wire round_up;
78         if (G && (R || S)) begin
79             round_up = 1'b1;
80         end else if (G && !(R || S) && mant_part[0]) begin
81             round_up = 1'b1;
82         end else begin
83             round_up = 1'b0;
84         end
85
86         wire [10:0] rounded_mant = {1'b0, mant_part} + round_up;
87         wire [9:0] final_mant;
88         wire [6:0] final_exp;
89
90         if (rounded_mant[10]) begin
91             final_mant = rounded_mant[10:1];
92             final_exp = exp_after_norm + 7'b1;
93         end else begin
94             final_mant = rounded_mant[9:0];
95             final_exp = exp_after_norm;
96         end
97
98         if (final_exp[6] || final_exp[5:0] > 6'b001111) begin
99             computed_C = {sign_A ^ sign_B, 5'b11111, 10'b0};
100        end else if (final_exp[5:0] < 6'b000001) begin
101            computed_C = {sign_A ^ sign_B, 5'b0, 10'b0};
102        end else begin

```

```
103         computed_C = {sign_A ^ sign_B, final_exp[4:0], final_mant};
104     end
105 end
106 end
107
108 always @(posedge clk or negedge rst_n) begin
109     if (!rst_n) begin
110         C <= 16'b0;
111         Valid <= 1'b0;
112     end else begin
113         if (Ready) begin
114             C <= computed_C;
115             Valid <= 1'b1;
116         end else begin
117             Valid <= 1'b0;
118         end
119     end
120 end
121 endmodule
```

Listing 3: vRTL251119112055_473.v

3 功能三：在聊天框进行代码纠错

将代码右键 Add to Chat，或者在 Chat 区添加文件。然后对模型提出修改任务和目标。这里我将任务一中补全的代码进行修改。

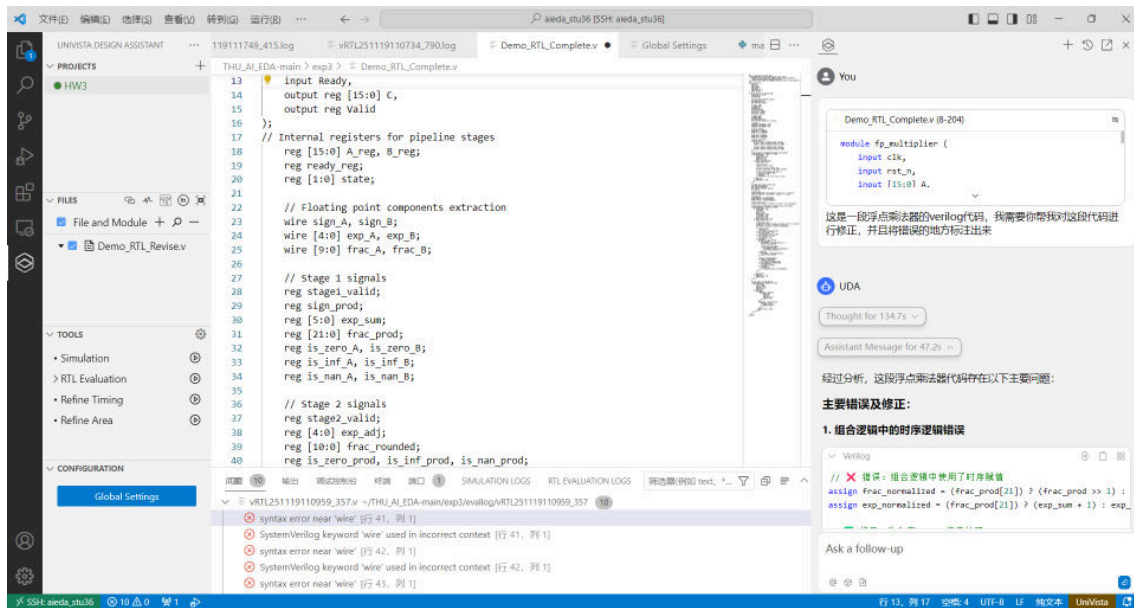


图 3: 聊天框纠错

将修改的结果插入到源代码中：

```

1 //Revised version
2
3
4 module fp_multiplier (
5     input clk,
6     input rst_n,
7     input [15:0] A,
8     input [15:0] B,
9     input Ready,
10    output reg [15:0] C,
11    output reg Valid
12);
13
14    // Internal registers for pipeline stages
15    reg [15:0] A_reg, B_reg;
16    reg ready_reg;
17    reg [1:0] state;
18
19    // Floating point components extraction
20    wire sign_A = A_reg[15];
21    wire [4:0] exp_A = A_reg[14:10];
22    wire [9:0] frac_A = A_reg[9:0];
23
24    wire sign_B = B_reg[15];
25    wire [4:0] exp_B = B_reg[14:10];
26    wire [9:0] frac_B = B_reg[9:0];
27
28    // Stage 1 signals
29    reg stage1_valid;
30    reg sign_prod;
31    reg [5:0] exp_sum;

```

```

32 reg [21:0] frac_prod;
33 reg is_zero_A, is_zero_B;
34 reg is_inf_A, is_inf_B;
35 reg is_nan_A, is_nan_B;
36
37 // Stage 2 signals
38 reg stage2_valid;
39 reg [5:0] exp_normalized;
40 reg [21:0] frac_normalized;
41 reg guard_bit, round_bit, sticky_bit;
42 reg round_up;
43 reg is_zero_prod, is_inf_prod, is_nan_prod;
44
45 // Constants
46 localparam [4:0] EXP_BIAS = 5'd15;
47 localparam [4:0] EXP_MAX = 5'd31;
48 localparam [4:0] EXP_MIN = 5'd0;
49
50 // Special case detection (combinational)
51 always @(*) begin
52     is_zero_A = (exp_A == 5'b0) && (frac_A == 10'b0);
53     is_inf_A = (exp_A == 5'b1111) && (frac_A == 10'b0);
54     is_nan_A = (exp_A == 5'b1111) && (frac_A != 10'b0);
55
56     is_zero_B = (exp_B == 5'b0) && (frac_B == 10'b0);
57     is_inf_B = (exp_B == 5'b1111) && (frac_B == 10'b0);
58     is_nan_B = (exp_B == 5'b1111) && (frac_B != 10'b0);
59 end
60
61 // Pipeline stage 1: Multiplication and special case handling
62 always @(posedge clk or negedge rst_n) begin
63     if (!rst_n) begin
64         stage1_valid <= 1'b0;
65         sign_prod <= 1'b0;
66         exp_sum <= 6'b0;
67         frac_prod <= 22'b0;
68         exp_normalized <= 6'b0;
69         frac_normalized <= 22'b0;
70         guard_bit <= 1'b0;
71         round_bit <= 1'b0;
72         sticky_bit <= 1'b0;
73         round_up <= 1'b0;
74     end else if (ready_reg) begin
75         stage1_valid <= 1'b1;
76
77         // Sign calculation
78         sign_prod <= sign_A ^ sign_B;
79
80         // Exponent calculation (subtract bias)
81         exp_sum <= {1'b0, exp_A} + {1'b0, exp_B} - EXP_BIAS;
82
83         // Fraction multiplication (with implicit 1)
84         if (is_zero_A || is_zero_B) begin
85             frac_prod <= 22'b0;
86         end else begin
87             frac_prod <= {1'b1, frac_A} * {1'b1, frac_B};
88         end
89
90         // Normalization logic (moved from combinational to sequential)
91         if (frac_prod[21]) begin
92             frac_normalized <= frac_prod >> 1;
93             exp_normalized <= exp_sum + 1;

```

```

94         end else begin
95             frac_normalized <= frac_prod;
96             exp_normalized <= exp_sum;
97         end
98
99         // Rounding bits calculation
100        guard_bit <= frac_normalized[10];
101        round_bit <= frac_normalized[9];
102        sticky_bit <= |frac_normalized[8:0];
103
104        // Round to nearest even
105        round_up <= guard_bit & (round_bit | sticky_bit | frac_normalized[11]);
106
107    end else begin
108        stage1_valid <= 1'b0;
109    end
110 end
111
112 // Pipeline stage 2: Rounding and final result assembly
113 always @(posedge clk or negedge rst_n) begin
114     if (!rst_n) begin
115         stage2_valid <= 1'b0;
116         C <= 16'b0;
117         Valid <= 1'b0;
118         is_zero_prod <= 1'b0;
119         is_inf_prod <= 1'b0;
120         is_nan_prod <= 1'b0;
121     end else if (stage1_valid) begin
122         stage2_valid <= 1'b1;
123
124         // Handle special cases
125         is_nan_prod <= is_nan_A | is_nan_B | (is_inf_A & is_zero_B) | (is_zero_A &
is_inf_B);
126         is_inf_prod <= (is_inf_A | is_inf_B) & ~is_nan_prod;
127         is_zero_prod <= (is_zero_A | is_zero_B) & ~is_nan_prod;
128
129         if (is_nan_prod) begin
130             // NaN result
131             C <= {sign_prod, 5'b11111, 10'b1};
132         end else if (is_inf_prod) begin
133             // Infinity result
134             C <= {sign_prod, 5'b11111, 10'b0};
135         end else if (is_zero_prod) begin
136             // Zero result
137             C <= {sign_prod, 15'b0};
138         end else begin
139             // Normal case: apply rounding with carry handling
140             reg [10:0] temp_frac_rounded;
141             reg [5:0] temp_exp_adj;
142
143             if (round_up) begin
144                 temp_frac_rounded = frac_normalized[21:11] + 1;
145                 temp_exp_adj = exp_normalized;
146
147                 // Handle carry from rounding
148                 if (temp_frac_rounded[10]) begin
149                     temp_frac_rounded = {1'b0, temp_frac_rounded[10:1]};
150                     temp_exp_adj = temp_exp_adj + 1;
151                 end
152             end else begin
153                 temp_frac_rounded = frac_normalized[21:11];
154                 temp_exp_adj = exp_normalized;

```

```

155         end
156
157         // Check for overflow/underflow
158         if (temp_exp_adj >= EXP_MAX) begin
159             // Overflow to infinity
160             C <= {sign_prod, 5'b11111, 10'b0};
161         end else if (temp_exp_adj <= EXP_MIN) begin
162             // Underflow to zero
163             C <= {sign_prod, 15'b0};
164         end else begin
165             // Normal result
166             C <= {sign_prod, temp_exp_adj[4:0], temp_frac_rounded[9:0]};
167         end
168     end
169
170     Valid <= 1'b1;
171 end else begin
172     stage2_valid <= 1'b0;
173     Valid <= 1'b0;
174 end
175 end
176
177 // Input register and state machine
178 always @(posedge clk or negedge rst_n) begin
179     if (!rst_n) begin
180         A_reg <= 16'b0;
181         B_reg <= 16'b0;
182         ready_reg <= 1'b0;
183         state <= 2'b00;
184     end else begin
185         case (state)
186             2'b00: begin // Idle state
187                 ready_reg <= 1'b0;
188                 if (Ready) begin
189                     A_reg <= A;
190                     B_reg <= B;
191                     ready_reg <= 1'b1;
192                     state <= 2'b01;
193                 end
194             end
195             2'b01: begin // Stage 1 processing
196                 ready_reg <= 1'b0;
197                 state <= 2'b10;
198             end
199             2'b10: begin // Stage 2 processing
200                 if (stage2_valid) begin
201                     state <= 2'b00;
202                 end
203             end
204             default: begin
205                 state <= 2'b00;
206                 ready_reg <= 1'b0;
207             end
208         endcase
209     end
210 end
211
212 endmodule

```

4 功能四：基于 Specification 的 RTL 生成

因为我想看看该 AI-EDA 工具的生成能力，因此这里采用的是助教在 HW1 中提供的 Zero Shot Prompt3，让其有更多可以自由发挥的设计空间。

```

1 Please help me generate a floating-point multiplier unit, including RTL code and TestBench.
2
3 ### Problem Description
4
5 ModelName: fp_multiplier
6
7 InputNode:
8
9 - clk: Clock signal (rising edge effective).
10 - rst_n: Active-low reset signal.
11 - A: 16-bit FP16 format input, operand one.
12 - B: 16-bit FP16 format input, operand two.
13 - Ready: Input data ready signal, high effective.
14
15 OutputNode:
16
17 - C: 16-bit FP16 format output.
18 - Valid: Device idle and result ready signal, high effective.
19
20 Specification:
21
22 - Support IEEE 754 Half-Precision Floating-Point Format (FP16):
23   - Handle all special cases (zero, infinity, NaN) as specified by IEEE 754.
24   - Ensure correct rounding to the nearest even value.
25 - Use valid and ready signals for handshake. When valid is high, the input operands A and B
   are ready. When ready is high, the result C is valid.
26 - TestBench should contain the following test conditions:
27   - Basic Multiplication: Positive numbers (e.g., 2.0 * 3.0); Negative numbers (e.g., -2.0 *
   3.0); Mixed signs (e.g., -2.0 * -3.0);
28   - Zero Multiplication: Multiplication involving zero (e.g., 0.0 * 2.0);
29   - Infinity Multiplication: Multiplication involving infinity (e.g., inf * 2.0);
30   - NaN Multiplication: Multiplication involving NaN (e.g., NaN * 2.0);
31   - Subnormal Numbers: Multiplication involving very small numbers (e.g., 5.96e-8 * 5.96e-8);
32   - Overflow and Underflow: Test cases that result in overflow or underflow;
33   - Edge Cases: Multiplication of maximum representable values (e.g., 65504.0 * 1.5);
   Multiplication of minimum representable values (e.g., 5.96e-8 * 5.96e-8);
34
35
36 ### RTL
37
38 ```verilog
39 module fp_multiplier
40 (
41     input wire clk,
42     input wire rst_n,
43     input wire [15:0] A,
44     input wire [15:0] B,
45     output wire [15:0] C,
46     input wire valid,
47     output wire ready
48 )
49 ```
50
51 ### TestBench
52
53 ```systemverilog
54 module fp_multiplier_tb;

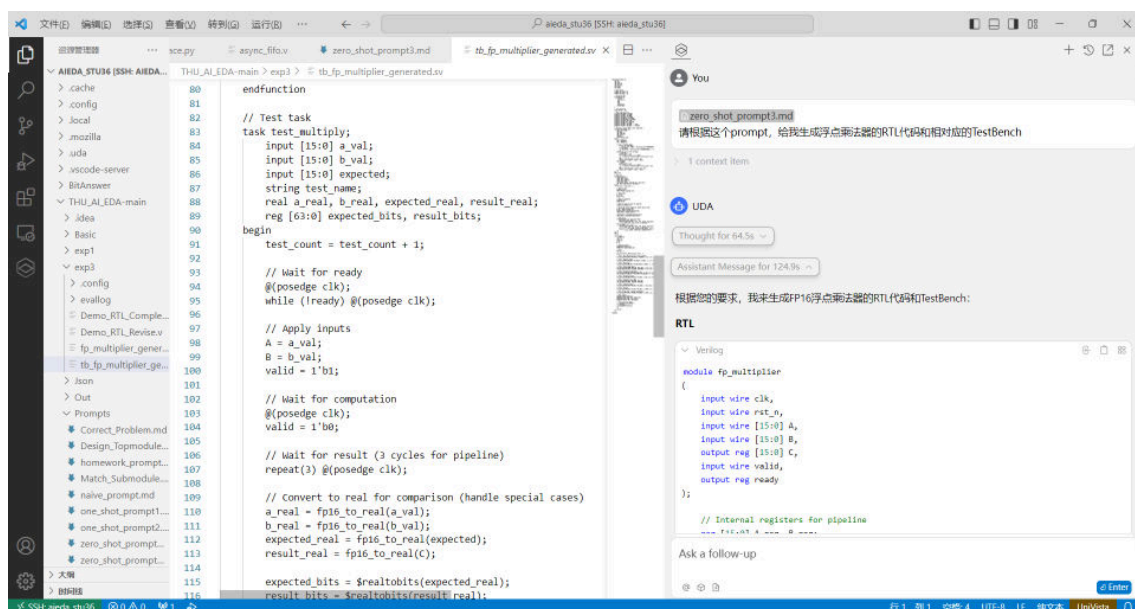
```

```

55 ***
56
57
58 ### Tips
59
60 Here is some tips that you need to follow:
61
62 1. When responding, please fill in RTL and TestBench respectively in the code box starting
    with "### RTL" and "### TestBench".
63
64 2. Use Verilog for RTL and SystemVerilog for TestBench.
65
66 3. We use the -g2005sv specification supported by iverilog for compilation. Please make sure
    that the verilog used can be supported.
67
68 4. Unpacked structs not supported in iverilog, do not use typedef struct.
69
70 5. When using the $bitstoreal function, please make sure that the parameter is 64bit.
71
72 6. Save the simulated waveform diagram with $dumpfile and $dumpvars instructions. Save the vcd
    file as Out/wave.vcd
73
74 7. In Testbench, please verify whether the function is correct. If you pass all the tests,
    please output TestSucceeded, otherwise you will output TestFailed.

```

将这个 prompt 放入 Chat 聊天框中，得到的生成结果如下。



```

1 module fp_multiplier
2 (
3     input wire clk,
4     input wire rst_n,
5     input wire [15:0] A,
6     input wire [15:0] B,
7     output reg [15:0] C,
8     input wire valid,
9     output reg ready
10 );
11
12 // Internal registers for pipeline
13 reg [15:0] A_reg, B_reg;
14 reg valid_reg;
15 reg [1:0] state;
16
17 // FP16 format: 1-bit sign, 5-bit exponent, 10-bit fraction
18 wire sign_a = A[15];
19 wire sign_b = B[15];
20 wire [4:0] exp_a = A[14:10];

```

```

21 wire [4:0] exp_b = B[14:10];
22 wire [9:0] frac_a = A[9:0];
23 wire [9:0] frac_b = B[9:0];
24
25 // Special case detection
26 wire a_is_zero = (exp_a == 5'b0) && (frac_a == 10'b0);
27 wire b_is_zero = (exp_b == 5'b0) && (frac_b == 10'b0);
28 wire a_is_inf = (exp_a == 5'b11111) && (frac_a == 10'b0);
29 wire b_is_inf = (exp_b == 5'b11111) && (frac_b == 10'b0);
30 wire a_is_nan = (exp_a == 5'b11111) && (frac_a != 10'b0);
31 wire b_is_nan = (exp_b == 5'b11111) && (frac_b != 10'b0);
32 wire a_is_subnormal = (exp_a == 5'b0) && (frac_a != 10'b0);
33 wire b_is_subnormal = (exp_b == 5'b0) && (frac_b != 10'b0);
34
35 // Pipeline stage 1: Special case handling and mantissa multiplication preparation
36 reg stage1_valid;
37 reg stage1_sign;
38 reg [5:0] stage1_exp_sum; // 6 bits for sum + bias adjustment
39 reg [20:0] stage1_mant_product; // 11x11 = 22 bits, but we use 21 for calculation
40 reg stage1_is_zero, stage1_is_inf, stage1_is_nan;
41
42 // Pipeline stage 2: Normalization and rounding
43 reg stage2_valid;
44 reg [15:0] stage2_result;
45
46 // FSM states
47 localparam IDLE = 2'b00;
48 localparam STAGE1 = 2'b01;
49 localparam STAGE2 = 2'b10;
50
51 always @(posedge clk or negedge rst_n) begin
52     if (!rst_n) begin
53         state <= IDLE;
54         ready <= 1'b1;
55         valid_reg <= 1'b0;
56         A_reg <= 16'b0;
57         B_reg <= 16'b0;
58         stage1_valid <= 1'b0;
59         stage2_valid <= 1'b0;
60         C <= 16'b0;
61     end else begin
62         case (state)
63             IDLE: begin
64                 ready <= 1'b1;
65                 if (valid && ready) begin
66                     A_reg <= A;
67                     B_reg <= B;
68                     valid_reg <= 1'b1;
69                     state <= STAGE1;
70                     ready <= 1'b0;
71
72                     // Stage 1 calculations
73                     stage1_sign <= sign_a ^ sign_b;
74
75                     // Handle special cases
76                     if (a_is_nan || b_is_nan) begin
77                         stage1_is_nan <= 1'b1;
78                         stage1_is_zero <= 1'b0;
79                         stage1_is_inf <= 1'b0;
80                         stage1_exp_sum <= 6'b0;
81                         stage1_mant_product <= 21'b0;
82                     end else if ((a_is_zero && b_is_inf) || (a_is_inf && b_is_zero)) begin

```

```

83         stage1_is_nan <= 1'b1;
84         stage1_is_zero <= 1'b0;
85         stage1_is_inf <= 1'b0;
86         stage1_exp_sum <= 6'b0;
87         stage1_mant_product <= 21'b0;
88     end else if (a_is_zero || b_is_zero) begin
89         stage1_is_zero <= 1'b1;
90         stage1_is_inf <= 1'b0;
91         stage1_is_nan <= 1'b0;
92         stage1_exp_sum <= 6'b0;
93         stage1_mant_product <= 21'b0;
94     end else if (a_is_inf || b_is_inf) begin
95         stage1_is_inf <= 1'b1;
96         stage1_is_zero <= 1'b0;
97         stage1_is_nan <= 1'b0;
98         stage1_exp_sum <= 6'b0;
99         stage1_mant_product <= 21'b0;
100    end else begin
101        // Normal multiplication
102        stage1_is_zero <= 1'b0;
103        stage1_is_inf <= 1'b0;
104        stage1_is_nan <= 1'b0;
105
106        // Add exponents and subtract bias (15 for FP16)
107        stage1_exp_sum <= {1'b0, exp_a} + {1'b0, exp_b} - 6'd15;
108
109        // Multiply mantissas (with hidden bit)
110        wire [10:0] mant_a = (a_is_subnormal) ? {1'b0, frac_a} : {1'b1,
frac_a};
111        wire [10:0] mant_b = (b_is_subnormal) ? {1'b0, frac_b} : {1'b1,
frac_b};
112        stage1_mant_product <= mant_a * mant_b;
113    end
114
115    stage1_valid <= 1'b1;
116    end
117    end
118
119    STAGE1: begin
120        if (stage1_valid) begin
121            // Stage 2: Normalization and rounding
122            if (stage1_is_nan) begin
123                stage2_result <= {stage1_sign, 5'b11111, 10'b1}; // Quiet NaN
124            end else if (stage1_is_inf) begin
125                stage2_result <= {stage1_sign, 5'b11111, 10'b0}; // Infinity
126            end else if (stage1_is_zero) begin
127                stage2_result <= {stage1_sign, 15'b0}; // Zero
128            end else begin
129                // Normal case: normalize and round
130                reg [4:0] final_exp;
131                reg [9:0] final_frac;
132                reg [20:0] normalized_mant;
133
134                // Check for overflow in exponent
135                if (stage1_exp_sum[5] || (stage1_exp_sum[4:0] > 5'd30)) begin
136                    // Overflow - return infinity
137                    stage2_result <= {stage1_sign, 5'b11111, 10'b0};
138                end else if (stage1_exp_sum[4:0] == 5'b0) begin
139                    // Underflow - return zero
140                    stage2_result <= {stage1_sign, 15'b0};
141                end else begin
142                    // Normal case

```

```

143         normalized_mant = stage1_mant_product;
144         final_exp = stage1_exp_sum[4:0];
145
146         // Normalize: product is in [1,4) range, so we may need to
shift
147         if (normalized_mant[20]) begin
148             normalized_mant = normalized_mant >> 1;
149             final_exp = final_exp + 1;
150
151             // Check for exponent overflow after normalization
152             if (final_exp > 5'd30) begin
153                 stage2_result <= {stage1_sign, 5'b11111, 10'b0};
154             end else begin
155                 // Round to nearest even (simplified)
156                 final_frac = normalized_mant[19:10];
157                 if (normalized_mant[9] && (normalized_mant[8:0] != 9'
b0 || normalized_mant[10])) begin
158                     final_frac = final_frac + 1;
159                     if (final_frac == 10'b0) begin // Carry occurred
160                         final_exp = final_exp + 1;
161                         if (final_exp > 5'd30) begin
162                             stage2_result <= {stage1_sign, 5'b11111,
10'b0};
163                         end else begin
164                             stage2_result <= {stage1_sign, final_exp,
final_frac};
165                         end
166                     end else begin
167                         stage2_result <= {stage1_sign, final_exp,
final_frac};
168                     end
169                 end else begin
170                     stage2_result <= {stage1_sign, final_exp,
final_frac};
171                 end
172             end
173         end else begin
174             // No normalization needed
175             final_frac = normalized_mant[18:9];
176             if (normalized_mant[8] && (normalized_mant[7:0] != 8'b0 ||
normalized_mant[9])) begin
177                 final_frac = final_frac + 1;
178                 if (final_frac == 10'b0) begin // Carry occurred
179                     final_exp = final_exp + 1;
180                     if (final_exp > 5'd30) begin
181                         stage2_result <= {stage1_sign, 5'b11111, 10'b0
};
182                     end else begin
183                         stage2_result <= {stage1_sign, final_exp,
final_frac};
184                     end
185                 end else begin
186                     stage2_result <= {stage1_sign, final_exp,
final_frac};
187                 end
188             end else begin
189                 stage2_result <= {stage1_sign, final_exp, final_frac};
190             end
191         end
192     end
193 end
194

```

```

195         stage2_valid <= 1'b1;
196         state <= STAGE2;
197     end
198 end
199
200     STAGE2: begin
201         if (stage2_valid) begin
202             C <= stage2_result;
203             stage2_valid <= 1'b0;
204             stage1_valid <= 1'b0;
205             valid_reg <= 1'b0;
206             state <= IDLE;
207         end
208     end
209 endcase
210 end
211 end
212
213 endmodule

```

Listing 4: fp_multiplier_generated.v

5 功能五: TestBench 生成及仿真结果

```

1 module fp_multiplier_tb;
2     reg clk;
3     reg rst_n;
4     reg [15:0] A;
5     reg [15:0] B;
6     wire [15:0] C;
7     reg valid;
8     wire ready;
9
10    // Test results
11    integer test_count = 0;
12    integer pass_count = 0;
13    integer fail_count = 0;
14
15    // Instantiate DUT
16    fp_multiplier dut (
17        .clk(clk),
18        .rst_n(rst_n),
19        .A(A),
20        .B(B),
21        .C(C),
22        .valid(valid),
23        .ready(ready)
24    );
25
26    // Clock generation
27    always #5 clk = ~clk;
28
29    // FP16 value definitions
30    parameter FP16_ZERO = 16'h0000;
31    parameter FP16_ONE = 16'h3C00;
32    parameter FP16_TWO = 16'h4000;
33    parameter FP16_THREE = 16'h4200;
34    parameter FP16_NEG_TWO = 16'hC000;
35    parameter FP16_NEG_THREE = 16'hC200;
36    parameter FP16_INF = 16'h7C00;

```

```

37 parameter FP16_NEG_INF = 16'hFC00;
38 parameter FP16_NAN = 16'h7E00;
39 parameter FP16_MAX = 16'h7BFF; // 65504
40 parameter FP16_MIN = 16'h0001; // -5.96e-8
41
42 // Function to convert FP16 to real (simplified)
43 function real fp16_to_real;
44     input [15:0] fp16;
45     reg sign;
46     reg [4:0] exp;
47     reg [9:0] frac;
48     real result;
49     integer i;
50     real fraction_value;
51 begin
52     sign = fp16[15];
53     exp = fp16[14:10];
54     frac = fp16[9:0];
55
56     if (exp == 5'b11111) begin
57         if (frac == 10'b0) begin
58             fp16_to_real = (sign) ? -$bitstoreal(64'h7FF0000000000000) : $bitstoreal(64'
h7FF00000000000000);
59         end else begin
60             fp16_to_real = $bitstoreal(64'h7FF8000000000000); // NaN
61         end
62     end else if (exp == 5'b0) then begin
63         // Subnormal
64         fraction_value = 0.0;
65         for (i = 0; i < 10; i = i + 1) begin
66             if (frac[9-i]) fraction_value = fraction_value + (2.0 ** (-i-1));
67         end
68         result = fraction_value * (2.0 ** -14);
69         fp16_to_real = (sign) ? -result : result;
70     end else begin
71         // Normal
72         fraction_value = 1.0;
73         for (i = 0; i < 10; i = i + 1) begin
74             if (frac[9-i]) fraction_value = fraction_value + (2.0 ** (-i-1));
75         end
76         result = fraction_value * (2.0 ** (exp - 15));
77         fp16_to_real = (sign) ? -result : result;
78     end
79 end
80 endfunction
81
82 // Test task
83 task test_multiply;
84     input [15:0] a_val;
85     input [15:0] b_val;
86     input [15:0] expected;
87     string test_name;
88     real a_real, b_real, expected_real, result_real;
89     reg [63:0] expected_bits, result_bits;
90 begin
91     test_count = test_count + 1;
92
93     // Wait for ready
94     @(posedge clk);
95     while (!ready) @(posedge clk);
96
97     // Apply inputs

```

```

98     A = a_val;
99     B = b_val;
100    valid = 1'b1;
101
102    // Wait for computation
103    @(posedge clk);
104    valid = 1'b0;
105
106    // Wait for result (3 cycles for pipeline)
107    repeat(3) @(posedge clk);
108
109    // Convert to real for comparison (handle special cases)
110    a_real = fp16_to_real(a_val);
111    b_real = fp16_to_real(b_val);
112    expected_real = fp16_to_real(expected);
113    result_real = fp16_to_real(C);
114
115    expected_bits = $realtobits(expected_real);
116    result_bits = $realtobits(result_real);
117
118    // Check result
119    if (expected_bits === result_bits) begin
120        pass_count = pass_count + 1;
121        $display("PASS: Test %0d - A=%h, B=%h, Got=%h, Expected=%h",
122            test_count, a_val, b_val, C, expected);
123    end else begin
124        fail_count = fail_count + 1;
125        $display("FAIL: Test %0d - A=%h, B=%h, Got=%h, Expected=%h",
126            test_count, a_val, b_val, C, expected);
127        $display("        A_real=%f, B_real=%f, Got_real=%f, Expected_real=%f",
128            a_real, b_real, result_real, expected_real);
129    end
130 end
131 endtask
132
133 initial begin
134     // Initialize
135     clk = 0;
136     rst_n = 0;
137     valid = 0;
138     A = 0;
139     B = 0;
140
141     // Generate waveform
142     $dumpfile("Out/wave.vcd");
143     $dumpvars(0, fp_multiplier_tb);
144
145     // Reset
146     #10 rst_n = 1;
147
148     $display("Starting FP16 Multiplier Tests...");
149
150     // Test 1: Basic Multiplication - Positive numbers
151     test_multiply(FP16_TW0, FP16_THREE, 16'h4400); // 2.0 * 3.0 = 6.0
152
153     // Test 2: Negative numbers
154     test_multiply(FP16_NEG_TW0, FP16_THREE, 16'hC400); // -2.0 * 3.0 = -6.0
155
156     // Test 3: Mixed signs
157     test_multiply(FP16_NEG_TW0, FP16_NEG_THREE, 16'h4400); // -2.0 * -3.0 = 6.0
158
159     // Test 4: Zero multiplication

```

```

160     test_multiply(FP16_ZERO, FP16_TWO, FP16_ZERO); // 0.0 * 2.0 = 0.0
161     test_multiply(FP16_TWO, FP16_ZERO, FP16_ZERO); // 2.0 * 0.0 = 0.0
162
163     // Test 5: Infinity multiplication
164     test_multiply(FP16_INF, FP16_TWO, FP16_INF); // inf * 2.0 = inf
165     test_multiply(FP16_NEG_INF, FP16_THREE, FP16_NEG_INF); // -inf * 3.0 = -inf
166
167     // Test 6: NaN multiplication
168     test_multiply(FP16_NAN, FP16_TWO, FP16_NAN); // NaN * 2.0 = NaN
169
170     // Test 7: Infinity * Zero = NaN
171     test_multiply(FP16_INF, FP16_ZERO, FP16_NAN);
172     test_multiply(FP16_ZERO, FP16_INF, FP16_NAN);
173
174     // Test 8: Subnormal numbers
175     test_multiply(FP16_MIN, FP16_MIN, FP16_ZERO); // Very small * very small = 0 (
underflow)
176
177     // Test 9: Overflow
178     test_multiply(FP16_MAX, FP16_TWO, FP16_INF); // 65504 * 2.0 = inf (overflow)
179
180     // Test 10: Edge case - maximum representable
181     test_multiply(FP16_MAX, FP16_ONE, FP16_MAX); // 65504 * 1.0 = 65504
182
183     // Final summary
184     $display("\nTest Summary:");
185     $display("Total Tests: %0d", test_count);
186     $display("Passed: %0d", pass_count);
187     $display("Failed: %0d", fail_count);
188
189     if (fail_count == 0) begin
190         $display("TestSuccesed");
191     end else begin
192         $display("TestFailed");
193     end
194
195     $finish;
196 end
197
198 endmodule

```

Listing 5: tb_fp_multiplier_generated.sv

在 FILES 中将这两个文件添加进来, 运行仿真。

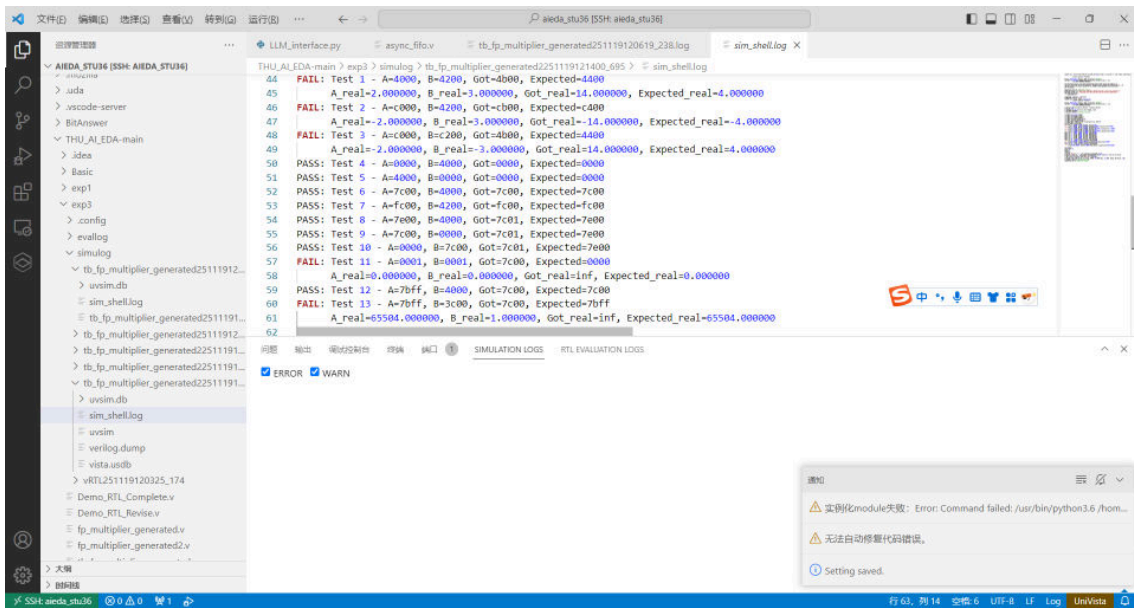


图 4: 第一次仿真结果

可以看到,这一版的代码可以运行,但是很多测试点的结果都不对。因此我们在聊天框将这个仿真结果输入给 AI,并让其帮我们修改代码。

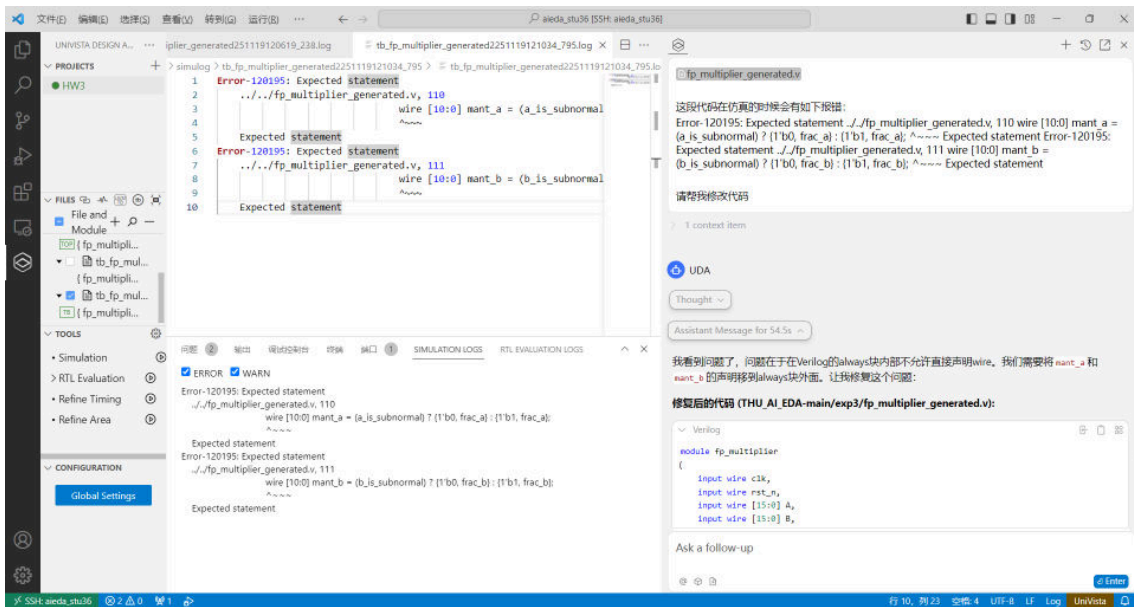


图 5: 结合仿真结果修改生成代码

生成的代码如下:

```

1 module fp_multiplier
2 (
3     input wire clk,
4     input wire rst_n,
5     input wire [15:0] A,
6     input wire [15:0] B,
7     output reg [15:0] C,
8     input wire valid,
9     output reg ready

```

```

10 );
11
12 // Internal registers for pipeline
13 reg [15:0] A_reg, B_reg;
14 reg valid_reg;
15 reg [1:0] state;
16
17 // FP16 format: 1-bit sign, 5-bit exponent, 10-bit fraction
18 wire sign_a = A[15];
19 wire sign_b = B[15];
20 wire [4:0] exp_a = A[14:10];
21 wire [4:0] exp_b = B[14:10];
22 wire [9:0] frac_a = A[9:0];
23 wire [9:0] frac_b = B[9:0];
24
25 // Special case detection
26 wire a_is_zero = (exp_a == 5'b0) && (frac_a == 10'b0);
27 wire b_is_zero = (exp_b == 5'b0) && (frac_b == 10'b0);
28 wire a_is_inf = (exp_a == 5'b11111) && (frac_a == 10'b0);
29 wire b_is_inf = (exp_b == 5'b11111) && (frac_b == 10'b0);
30 wire a_is_nan = (exp_a == 5'b11111) && (frac_a != 10'b0);
31 wire b_is_nan = (exp_b == 5'b11111) && (frac_b != 10'b0);
32 wire a_is_subnormal = (exp_a == 5'b0) && (frac_a != 10'b0);
33 wire b_is_subnormal = (exp_b == 5'b0) && (frac_b != 10'b0);
34
35 // Mantissa calculation wires
36 wire [10:0] mant_a = (a_is_subnormal || a_is_zero) ? {1'b0, frac_a} : {1'b1, frac_a};
37 wire [10:0] mant_b = (b_is_subnormal || b_is_zero) ? {1'b0, frac_b} : {1'b1, frac_b};
38
39 // Pipeline stage 1: Special case handling and mantissa multiplication preparation
40 reg stage1_valid;
41 reg stage1_sign;
42 reg [5:0] stage1_exp_sum; // 6 bits for sum + bias adjustment
43 reg [21:0] stage1_mant_product; // 11x11 = 22 bits
44 reg stage1_is_zero, stage1_is_inf, stage1_is_nan;
45
46 // Pipeline stage 2: Normalization and rounding
47 reg stage2_valid;
48 reg [15:0] stage2_result;
49
50 // FSM states
51 localparam IDLE = 2'b00;
52 localparam STAGE1 = 2'b01;
53 localparam STAGE2 = 2'b10;
54
55 always @(posedge clk or negedge rst_n) begin
56     if (!rst_n) begin
57         state <= IDLE;
58         ready <= 1'b1;
59         valid_reg <= 1'b0;
60         A_reg <= 16'b0;
61         B_reg <= 16'b0;
62         stage1_valid <= 1'b0;
63         stage2_valid <= 1'b0;
64         C <= 16'b0;
65     end else begin
66         case (state)
67             IDLE: begin
68                 ready <= 1'b1;
69                 if (valid && ready) begin
70                     A_reg <= A;
71                     B_reg <= B;

```

```

72         valid_reg <= 1'b1;
73         state <= STAGE1;
74         ready <= 1'b0;
75
76         // Stage 1 calculations
77         stage1_sign <= sign_a ^ sign_b;
78
79         // Handle special cases
80         if (a_is_nan || b_is_nan) begin
81             stage1_is_nan <= 1'b1;
82             stage1_is_zero <= 1'b0;
83             stage1_is_inf <= 1'b0;
84             stage1_exp_sum <= 6'b0;
85             stage1_mant_product <= 22'b0;
86         end else if ((a_is_zero && b_is_inf) || (a_is_inf && b_is_zero)) begin
87             stage1_is_nan <= 1'b1;
88             stage1_is_zero <= 1'b0;
89             stage1_is_inf <= 1'b0;
90             stage1_exp_sum <= 6'b0;
91             stage1_mant_product <= 22'b0;
92         end else if (a_is_zero || b_is_zero) begin
93             stage1_is_zero <= 1'b1;
94             stage1_is_inf <= 1'b0;
95             stage1_is_nan <= 1'b0;
96             stage1_exp_sum <= 6'b0;
97             stage1_mant_product <= 22'b0;
98         end else if (a_is_inf || b_is_inf) begin
99             stage1_is_inf <= 1'b1;
100            stage1_is_zero <= 1'b0;
101            stage1_is_nan <= 1'b0;
102            stage1_exp_sum <= 6'b0;
103            stage1_mant_product <= 22'b0;
104        end else begin
105            // Normal multiplication
106            stage1_is_zero <= 1'b0;
107            stage1_is_inf <= 1'b0;
108            stage1_is_nan <= 1'b0;
109
110            // Add exponents and subtract bias (15 for FP16)
111            // For normal numbers: exp = stored_exp - 15
112            // So product_exp = (exp_a - 15) + (exp_b - 15) + 15 = exp_a +
exp_b - 15
113
114            stage1_exp_sum <= {1'b0, exp_a} + {1'b0, exp_b} - 6'd15;
115
116            // Multiply mantissas (with hidden bit)
117            stage1_mant_product <= mant_a * mant_b;
118        end
119
120        stage1_valid <= 1'b1;
121    end
122
123    STAGE1: begin
124        if (stage1_valid) begin
125            // Stage 2: Normalization and rounding
126            if (stage1_is_nan) begin
127                // Return canonical NaN
128                stage2_result <= {stage1_sign, 5'b11111, 10'b1000000000};
129            end else if (stage1_is_inf) begin
130                stage2_result <= {stage1_sign, 5'b11111, 10'b0}; // Infinity
131            end else if (stage1_is_zero) begin
132                stage2_result <= {stage1_sign, 15'b0}; // Zero

```

```

133         end else begin
134             // Normal case: normalize and round
135             reg [5:0] temp_exp;
136             reg [9:0] temp_frac;
137             reg [21:0] temp_mant;
138
139             temp_exp = stage1_exp_sum;
140             temp_mant = stage1_mant_product;
141
142             // Normalize: product is in [1,4) range, so we may need to shift
143             if (temp_mant[21]) begin
144                 // Product >= 2, need to shift right
145                 temp_mant = temp_mant >> 1;
146                 temp_exp = temp_exp + 1;
147             end
148
149             // Check for overflow/underflow
150             if (temp_exp[5] || temp_exp[4:0] > 5'd30) begin
151                 // Overflow - return infinity
152                 stage2_result <= {stage1_sign, 5'b11111, 10'b0};
153             end else if (temp_exp[4:0] == 5'b0) begin
154                 // Underflow - return zero
155                 stage2_result <= {stage1_sign, 15'b0};
156             end else begin
157                 // Extract fraction (ignore the hidden bit at position 20)
158                 temp_frac = temp_mant[20:11];
159
160                 // Round to nearest even
161                 if (temp_mant[10] && (temp_mant[9:0] != 10'b0 || temp_mant
162 [11])) begin
163                     temp_frac = temp_frac + 1;
164                     if (temp_frac == 10'b0) begin // Carry to exponent
165                         temp_exp = temp_exp + 1;
166                         if (temp_exp > 5'd30) begin
167                             stage2_result <= {stage1_sign, 5'b11111, 10'b0};
168                             end else begin
169                                 stage2_result <= {stage1_sign, temp_exp[4:0],
170 temp_frac};
171
172                                 end
173                             end else begin
174                                 stage2_result <= {stage1_sign, temp_exp[4:0], temp_frac};
175                             end
176                         end
177                     end
178
179                     stage2_valid <= 1'b1;
180                     state <= STAGE2;
181                 end
182             end
183
184             STAGE2: begin
185                 if (stage2_valid) begin
186                     C <= stage2_result;
187                     stage2_valid <= 1'b0;
188                     stage1_valid <= 1'b0;
189                     valid_reg <= 1'b0;
190                     state <= IDLE;
191                 end

```

```

192         end
193     endcase
194 end
195 end
196
197 endmodule

```

Listing 6: fp_multiplier_generated3.sv

```

1 module fp_multiplier_tb;
2     reg clk;
3     reg rst_n;
4     reg [15:0] A;
5     reg [15:0] B;
6     wire [15:0] C;
7     reg valid;
8     wire ready;
9
10    // Test results
11    integer test_count = 0;
12    integer pass_count = 0;
13    integer fail_count = 0;
14
15    // Instantiate DUT
16    fp_multiplier dut (
17        .clk(clk),
18        .rst_n(rst_n),
19        .A(A),
20        .B(B),
21        .C(C),
22        .valid(valid),
23        .ready(ready)
24    );
25
26    // Clock generation
27    always #5 clk = ~clk;
28
29    // FP16 value definitions
30    parameter FP16_ZERO = 16'h0000;
31    parameter FP16_ONE = 16'h3C00;
32    parameter FP16_TWO = 16'h4000;
33    parameter FP16_THREE = 16'h4200;
34    parameter FP16_NEG_TWO = 16'hC000;
35    parameter FP16_NEG_THREE = 16'hC200;
36    parameter FP16_INF = 16'h7C00;
37    parameter FP16_NEG_INF = 16'hFC00;
38    parameter FP16_NAN = 16'h7E00;
39    parameter FP16_MAX = 16'h7BFF; // 65504
40    parameter FP16_MIN = 16'h0001; // -5.96e-8
41    parameter FP16_SIX = 16'h4600; // 6.0
42
43    // Function to convert FP16 to real (simplified)
44    function real fp16_to_real;
45        input [15:0] fp16;
46        reg sign;
47        reg [4:0] exp;
48        reg [9:0] frac;
49        real result;
50        integer i;
51        real fraction_value;
52    begin
53        sign = fp16[15];

```

```

54     exp = fp16[14:10];
55     frac = fp16[9:0];
56
57     if (exp == 5'b11111) begin
58         if (frac == 10'b0) begin
59             fp16_to_real = (sign) ? -$bitstoreal(64'h7FF0000000000000) : $bitstoreal(64'
60             h7FF00000000000000);
61         end else begin
62             fp16_to_real = $bitstoreal(64'h7FF8000000000000); // NaN
63         end
64     end else if (exp == 5'b0) begin
65         // Subnormal
66         fraction_value = 0.0;
67         for (i = 0; i < 10; i = i + 1) begin
68             if (frac[9-i]) fraction_value = fraction_value + (2.0 ** (-i-1));
69         end
70         result = fraction_value * (2.0 ** -14);
71         fp16_to_real = (sign) ? -result : result;
72     end else begin
73         // Normal
74         fraction_value = 1.0;
75         for (i = 0; i < 10; i = i + 1) begin
76             if (frac[9-i]) fraction_value = fraction_value + (2.0 ** (-i-1));
77         end
78         result = fraction_value * (2.0 ** (exp - 15));
79         fp16_to_real = (sign) ? -result : result;
80     end
81 endfunction
82
83 // Function to check if two FP16 values are equal (including special cases)
84 function bit fp16_equal;
85     input [15:0] a;
86     input [15:0] b;
87     begin
88         // For NaN, we consider any NaN equal to any other NaN
89         if ((a[14:10] == 5'b11111 && a[9:0] != 10'b0) &&
90             (b[14:10] == 5'b11111 && b[9:0] != 10'b0)) begin
91             fp16_equal = 1'b1;
92         end else begin
93             fp16_equal = (a == b);
94         end
95     end
96 endfunction
97
98 // Test task
99 task test_multiply;
100     input [15:0] a_val;
101     input [15:0] b_val;
102     input [15:0] expected;
103     string test_name;
104     real a_real, b_real, expected_real, result_real;
105     begin
106         test_count = test_count + 1;
107
108         // Wait for ready
109         @(posedge clk);
110         while (!ready) @(posedge clk);
111
112         // Apply inputs
113         A = a_val;
114         B = b_val;

```

```

115     valid = 1'b1;
116
117     // Wait for computation
118     @(posedge clk);
119     valid = 1'b0;
120
121     // Wait for result (3 cycles for pipeline)
122     repeat(3) @(posedge clk);
123
124     // Convert to real for debug
125     a_real = fp16_to_real(a_val);
126     b_real = fp16_to_real(b_val);
127     expected_real = fp16_to_real(expected);
128     result_real = fp16_to_real(C);
129
130     // Check result using fp16_equal function for proper NaN handling
131     if (fp16_equal(C, expected)) begin
132         pass_count = pass_count + 1;
133         $display("PASS: Test %0d - A=%h, B=%h, Got=%h, Expected=%h",
134             test_count, a_val, b_val, C, expected);
135     end else begin
136         fail_count = fail_count + 1;
137         $display("FAIL: Test %0d - A=%h, B=%h, Got=%h, Expected=%h",
138             test_count, a_val, b_val, C, expected);
139         $display("      A_real=%f, B_real=%f, Got_real=%f, Expected_real=%f",
140             a_real, b_real, result_real, expected_real);
141     end
142 end
143 endtask
144
145 initial begin
146     // Initialize
147     clk = 0;
148     rst_n = 0;
149     valid = 0;
150     A = 0;
151     B = 0;
152
153     // Generate waveform
154     $dumpfile("Out/wave.vcd");
155     $dumpvars(0, fp_multiplier_tb);
156
157     // Reset
158     #10 rst_n = 1;
159
160     $display("Starting FP16 Multiplier Tests...");
161
162     // Test 1: Basic Multiplication - Positive numbers
163     test_multiply(FP16_TWO, FP16_THREE, FP16_SIX); // 2.0 * 3.0 = 6.0
164
165     // Test 2: Negative numbers
166     test_multiply(FP16_NEG_TWO, FP16_THREE, 16'hC600); // -2.0 * 3.0 = -6.0
167
168     // Test 3: Mixed signs
169     test_multiply(FP16_NEG_TWO, FP16_NEG_THREE, FP16_SIX); // -2.0 * -3.0 = 6.0
170
171     // Test 4: Zero multiplication
172     test_multiply(FP16_ZERO, FP16_TWO, FP16_ZERO); // 0.0 * 2.0 = 0.0
173     test_multiply(FP16_TWO, FP16_ZERO, FP16_ZERO); // 2.0 * 0.0 = 0.0
174
175     // Test 5: Infinity multiplication
176     test_multiply(FP16_INF, FP16_TWO, FP16_INF); // inf * 2.0 = inf

```

```

177     test_multiply(FP16_NEG_INF, FP16_THREE, FP16_NEG_INF); // -inf * 3.0 = -inf
178
179     // Test 6: NaN multiplication
180     test_multiply(FP16_NAN, FP16_TWO, FP16_NAN); // NaN * 2.0 = NaN
181
182     // Test 7: Infinity * Zero = NaN
183     test_multiply(FP16_INF, FP16_ZERO, FP16_NAN);
184     test_multiply(FP16_ZERO, FP16_INF, FP16_NAN);
185
186     // Test 8: Subnormal numbers - expect underflow to zero
187     test_multiply(FP16_MIN, FP16_MIN, FP16_ZERO); // Very small * very small = 0 (
underflow)
188
189     // Test 9: Overflow
190     test_multiply(FP16_MAX, FP16_TWO, FP16_INF); // 65504 * 2.0 = inf (overflow)
191
192     // Test 10: Edge case - maximum representable
193     test_multiply(FP16_MAX, FP16_ONE, FP16_MAX); // 65504 * 1.0 = 65504
194
195     // Additional test: 1.0 * 1.0 = 1.0
196     test_multiply(FP16_ONE, FP16_ONE, FP16_ONE);
197
198     // Final summary
199     $display("\nTest Summary:");
200     $display("Total Tests: %0d", test_count);
201     $display("Passed: %0d", pass_count);
202     $display("Failed: %0d", fail_count);
203
204     if (fail_count == 0) begin
205         $display("TestSucceeded");
206     end else begin
207         $display("TestFailed");
208     end
209
210     $finish;
211 end
212
213 endmodule

```

Listing 7: vRTL251119114743_749.v

6 功能六：时序和面积优化

除了以上的功能，我还尝试了一下 Explore QoR 功能，对刚刚生成的浮点乘法器做面积、时序优化。

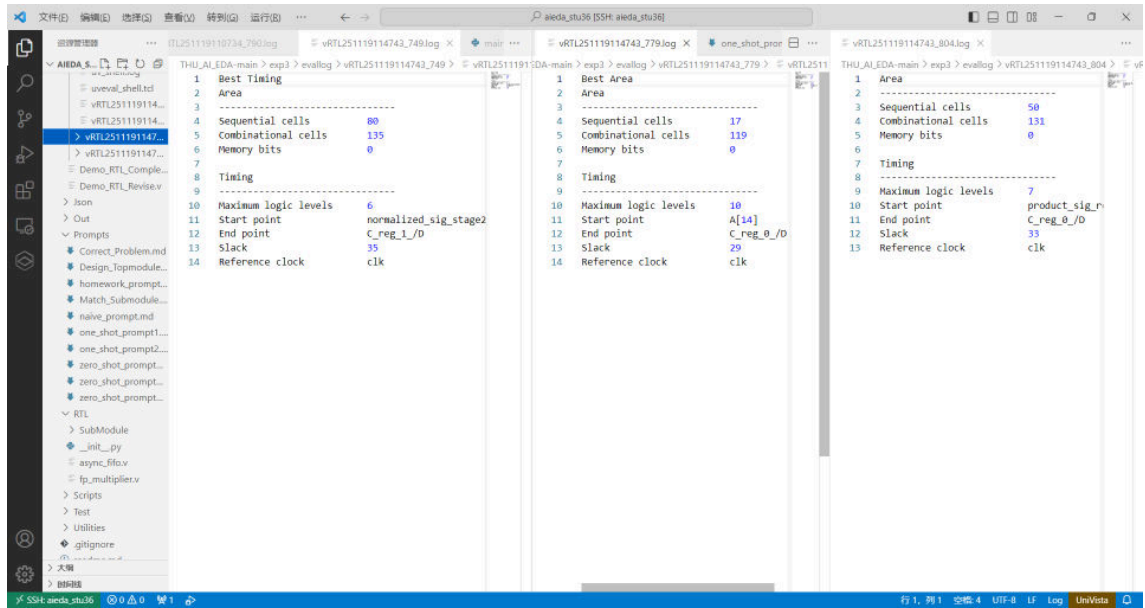


图 6: 时序和面积优化

最优时序和最优面积的代码如下：

```

1 module fp_multiplier (
2     input clk,
3     input rst_n,
4     input [15:0] A,
5     input [15:0] B,
6     input Ready,
7     output reg [15:0] C,
8     output reg Valid
9 );
10
11 // Internal registers and wires
12 reg [15:0] computed_C;
13
14 // Input decomposition
15 wire sign_A;
16 wire [4:0] exp_A;
17 wire [9:0] mant_A;
18 wire sign_B;
19 wire [4:0] exp_B;
20 wire [9:0] mant_B;
21
22 // Special case detection
23 wire is_nan_A;
24 wire is_inf_A;
25 wire is_zero_A;
26 wire is_nan_B;
27 wire is_inf_B;
28 wire is_zero_B;
29
30 // Computation signals - stage 1
31 wire [10:0] sig_A;

```

```

32 wire [10:0] sig_B;
33 wire [21:0] product_sig;
34 wire [5:0] exp_A_adj;
35 wire [5:0] exp_B_adj;
36 wire [6:0] exp_sum;
37 wire [6:0] exp_prod;
38
39 // Normalization signals - stage 2
40 wire [4:0] shift_amount;
41 wire [21:0] normalized_sig;
42 wire [6:0] exp_after_norm;
43
44 // Rounding signals - stage 3
45 wire [9:0] mant_part;
46 wire G;
47 wire R;
48 wire S;
49 wire round_up;
50 wire [10:0] rounded_mant;
51 wire [9:0] final_mant;
52 wire [6:0] final_exp;
53
54 // Pipeline registers
55 reg [1:0] special_case_stage1;
56 reg sign_prod_stage1;
57 reg [21:0] product_sig_stage1;
58 reg [6:0] exp_prod_stage1;
59
60 reg [1:0] special_case_stage2;
61 reg sign_prod_stage2;
62 reg [21:0] normalized_sig_stage2;
63 reg [6:0] exp_after_norm_stage2;
64
65 // Assign input decomposition
66 assign sign_A = A[15];
67 assign exp_A = A[14:10];
68 assign mant_A = A[9:0];
69 assign sign_B = B[15];
70 assign exp_B = B[14:10];
71 assign mant_B = B[9:0];
72
73 // Special case detection
74 assign is_nan_A = (exp_A == 5'b11111) && (mant_A != 10'b0);
75 assign is_inf_A = (exp_A == 5'b11111) && (mant_A == 10'b0);
76 assign is_zero_A = (exp_A == 5'b0) && (mant_A == 10'b0);
77 assign is_nan_B = (exp_B == 5'b11111) && (mant_B != 10'b0);
78 assign is_inf_B = (exp_B == 5'b11111) && (mant_B == 10'b0);
79 assign is_zero_B = (exp_B == 5'b0) && (mant_B == 10'b0);
80
81 // Stage 1: Basic computation
82 assign sig_A = (exp_A == 5'b0) ? {1'b0, mant_A} : {1'b1, mant_A};
83 assign sig_B = (exp_B == 5'b0) ? {1'b0, mant_B} : {1'b1, mant_B};
84 assign product_sig = sig_A * sig_B;
85 assign exp_A_adj = (exp_A == 5'b0) ? 6'b100001 : {1'b0, exp_A};
86 assign exp_B_adj = (exp_B == 5'b0) ? 6'b100001 : {1'b0, exp_B};
87 assign exp_sum = exp_A_adj + exp_B_adj;
88 assign exp_prod = exp_sum - 7'b0011110;
89
90 // Stage 1 pipeline register
91 always @(posedge clk or negedge rst_n) begin
92     if (!rst_n) begin
93         special_case_stage1 <= 2'b00;

```

```

94     sign_prod_stage1 <= 1'b0;
95     product_sig_stage1 <= 22'b0;
96     exp_prod_stage1 <= 7'b0;
97 end else if (Ready) begin
98     // Encode special cases
99     if (is_nan_A || is_nan_B) begin
100         special_case_stage1 <= 2'b01; // NaN
101     end else if ((is_inf_A && is_zero_B) || (is_zero_A && is_inf_B)) begin
102         special_case_stage1 <= 2'b01; // NaN
103     end else if (is_inf_A || is_inf_B) begin
104         special_case_stage1 <= 2'b10; // Infinity
105     end else if (is_zero_A || is_zero_B) begin
106         special_case_stage1 <= 2'b11; // Zero
107     end else begin
108         special_case_stage1 <= 2'b00; // Normal case
109     end
110
111     sign_prod_stage1 <= sign_A ^ sign_B;
112     product_sig_stage1 <= product_sig;
113     exp_prod_stage1 <= exp_prod;
114 end
115 end
116
117 // Stage 2: Normalization
118 assign shift_amount = (product_sig_stage1[21]) ? 5'b0 :
119                       (product_sig_stage1[20]) ? 5'b1 :
120                       (product_sig_stage1[19]) ? 5'b10 :
121                       (product_sig_stage1[18]) ? 5'b11 : 5'b0;
122
123 assign normalized_sig = product_sig_stage1 << shift_amount;
124 assign exp_after_norm = exp_prod_stage1 - {2'b0, shift_amount};
125
126 // Stage 2 pipeline register
127 always @(posedge clk or negedge rst_n) begin
128     if (!rst_n) begin
129         special_case_stage2 <= 2'b00;
130         sign_prod_stage2 <= 1'b0;
131         normalized_sig_stage2 <= 22'b0;
132         exp_after_norm_stage2 <= 7'b0;
133     end else begin
134         special_case_stage2 <= special_case_stage1;
135         sign_prod_stage2 <= sign_prod_stage1;
136         normalized_sig_stage2 <= normalized_sig;
137         exp_after_norm_stage2 <= exp_after_norm;
138     end
139 end
140
141 // Stage 3: Rounding and final computation
142 assign mant_part = normalized_sig_stage2[20:11];
143 assign G = normalized_sig_stage2[10];
144 assign R = normalized_sig_stage2[9];
145 assign S = |normalized_sig_stage2[8:0];
146
147 assign round_up = (G && (R || S)) ? 1'b1 :
148                  (G && !(R || S) && mant_part[0]) ? 1'b1 : 1'b0;
149
150 assign rounded_mant = {1'b0, mant_part} + round_up;
151 assign final_mant = rounded_mant[10:1] ? rounded_mant[10:1] : rounded_mant[9:0];
152 assign final_exp = rounded_mant[10] ? (exp_after_norm_stage2 + 7'b1) :
153                  exp_after_norm_stage2;
154
155 // Final result computation

```

```

155     always @(*) begin
156         computed_C = 16'b0;
157
158         case (special_case_stage2)
159             2'b01: computed_C = {1'b0, 5'b11111, 10'b1}; // NaN
160             2'b10: computed_C = {sign_prod_stage2, 5'b11111, 10'b0}; // Infinity
161             2'b11: computed_C = {sign_prod_stage2, 5'b0, 10'b0}; // Zero
162             default: begin
163                 if (final_exp[6] || final_exp[5:0] > 6'b001111) begin
164                     computed_C = {sign_prod_stage2, 5'b11111, 10'b0};
165                 end else if (final_exp[5:0] < 6'b000001) begin
166                     computed_C = {sign_prod_stage2, 5'b0, 10'b0};
167                 end else begin
168                     computed_C = {sign_prod_stage2, final_exp[4:0], final_mant};
169                 end
170             end
171         endcase
172     end
173
174     // Output register
175     always @(posedge clk or negedge rst_n) begin
176         if (!rst_n) begin
177             C <= 16'b0;
178             Valid <= 1'b0;
179         end else begin
180             C <= computed_C;
181             Valid <= Ready; // Valid follows Ready with pipeline delay
182         end
183     end
184
185 endmodule

```

Listing 8: vRTL251119114743_779.v

```

1 module fp_multiplier (
2     input clk,
3     input rst_n,
4     input [15:0] A,
5     input [15:0] B,
6     input Ready,
7     output reg [15:0] C,
8     output reg Valid
9 );
10
11 // Internal registers and wires
12 reg [15:0] computed_C;
13
14 // Input decomposition
15 wire sign_A = A[15];
16 wire [4:0] exp_A = A[14:10];
17 wire [9:0] mant_A = A[9:0];
18 wire sign_B = B[15];
19 wire [4:0] exp_B = B[14:10];
20 wire [9:0] mant_B = B[9:0];
21
22 // Special case detection
23 wire is_nan_A = (exp_A == 5'b11111) && (mant_A != 10'b0);
24 wire is_inf_A = (exp_A == 5'b11111) && (mant_A == 10'b0);
25 wire is_zero_A = (exp_A == 5'b0) && (mant_A == 10'b0);
26 wire is_nan_B = (exp_B == 5'b11111) && (mant_B != 10'b0);
27 wire is_inf_B = (exp_B == 5'b11111) && (mant_B == 10'b0);
28 wire is_zero_B = (exp_B == 5'b0) && (mant_B == 10'b0);

```

```

29
30 // Shared computation signals
31 wire [10:0] sig_A = (exp_A == 5'b0) ? {1'b0, mant_A} : {1'b1, mant_A};
32 wire [10:0] sig_B = (exp_B == 5'b0) ? {1'b0, mant_B} : {1'b1, mant_B};
33 wire [21:0] product_sig = sig_A * sig_B;
34 wire [5:0] exp_A_adj = (exp_A == 5'b0) ? 6'b100001 : {1'b0, exp_A};
35 wire [5:0] exp_B_adj = (exp_B == 5'b0) ? 6'b100001 : {1'b0, exp_B};
36 wire [6:0] exp_sum = exp_A_adj + exp_B_adj;
37 wire [6:0] exp_prod = exp_sum - 7'b0011110;
38
39 // Area-optimized normalization using shared logic
40 wire [4:0] shift_amount;
41 wire [21:0] normalized_sig;
42
43 // Optimized shift amount calculation using priority encoder style
44 assign shift_amount = (~product_sig[21] & ~product_sig[20] & ~product_sig[19] & ~
45 product_sig[18]) ? 5'b0 :
46 (~product_sig[21] & ~product_sig[20] & ~product_sig[19] & product_sig
47 [18]) ? 5'b11 :
48 (~product_sig[21] & ~product_sig[20] & product_sig[19]) ? 5'b10 :
49 (~product_sig[21] & product_sig[20]) ? 5'b1 : 5'b0;
50
51 // Shared barrel shifter
52 assign normalized_sig = product_sig << shift_amount;
53
54 // Shared rounding logic
55 wire [9:0] mant_part = normalized_sig[20:11];
56 wire G = normalized_sig[10];
57 wire R = normalized_sig[9];
58 wire S = |normalized_sig[8:0];
59 wire round_up = (G && (R || S)) || (G && !(R || S) && mant_part[0]);
60 wire [10:0] rounded_mant = {1'b0, mant_part} + round_up;
61
62 wire [9:0] final_mant = rounded_mant[10] ? rounded_mant[10:1] : rounded_mant[9:0];
63 wire [6:0] final_exp = rounded_mant[10] ? (exp_after_norm + 7'b1) : exp_after_norm;
64
65 // Combined result computation
66 always @(*) begin
67     computed_C = 16'b0;
68
69     if (is_nan_A || is_nan_B) begin
70         computed_C = {1'b0, 5'b11111, 10'b1};
71     end else if ((is_inf_A && is_zero_B) || (is_zero_A && is_inf_B)) begin
72         computed_C = {1'b0, 5'b11111, 10'b1};
73     end else if (is_inf_A || is_inf_B) begin
74         computed_C = {sign_A ^ sign_B, 5'b11111, 10'b0};
75     end else if (is_zero_A || is_zero_B) begin
76         computed_C = {sign_A ^ sign_B, 5'b0, 10'b0};
77     end else begin
78         if (final_exp[6] || final_exp[5:0] > 6'b001111) begin
79             computed_C = {sign_A ^ sign_B, 5'b11111, 10'b0};
80         end else if (final_exp[5:0] < 6'b000001) begin
81             computed_C = {sign_A ^ sign_B, 5'b0, 10'b0};
82         end else begin
83             computed_C = {sign_A ^ sign_B, final_exp[4:0], final_mant};
84         end
85     end
86 end
87
88 // Output register

```

```
89     always @(posedge clk or negedge rst_n) begin
90         if (!rst_n) begin
91             C <= 16'b0;
92             Valid <= 1'b0;
93         end else begin
94             if (Ready) begin
95                 C <= computed_C;
96                 Valid <= 1'b1;
97             end else begin
98                 Valid <= 1'b0;
99             end
100         end
101     end
102
103 endmodule
```

Listing 9: fp_multiplier_generated3.sv

从代码可以看到, 对于优化时序, 其采用了四级流水线设计; 而对于优化面积, 其降低了时序逻辑和组合逻辑的个数。

7 对合见工软 UDA 的使用评价, 修改建议

做完整个任务后, 我深深感受到了这个 UDA 工具的强大。其功能十分完善, 纠错、补全代码、QoR 探索、一站式验证调试等已经能满足大部分设计者基本的设计任务, 同时使用手册还提供了更加高阶的使用操作。并且内嵌 AI 模型, 可以在设计中的 Chat 聊天框中和 AI 对话, 非常的方便且灵活。

除此之外, 一些细节的设计也非常到位。比如在某个代码中做了改错的操作, 会在代码开头显示过对这代码有过 UDA 的什么操作; 在 Chat 聊天框中可以一键插入代码、保存代码、新旧代码对比, 非常好用。

至于修改建议, 可以在设置中的 prompt add 里添加一个直接读取文件的功能; 同时在修改代码时, 可以自动跑一遍仿真, 将仿真结果作为 AI 下一次修改意见; 以及在 FILES 区域添加文件的时候, 可能是有小 bug, 有几次点进去添加文件, 结果文件夹中的代码文件根本没有显示。